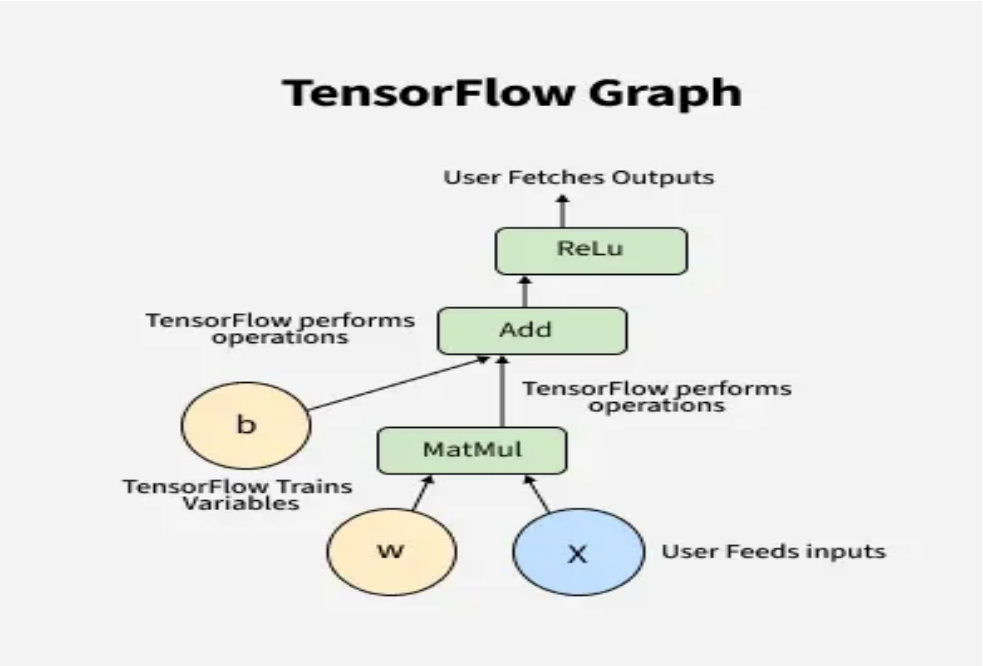
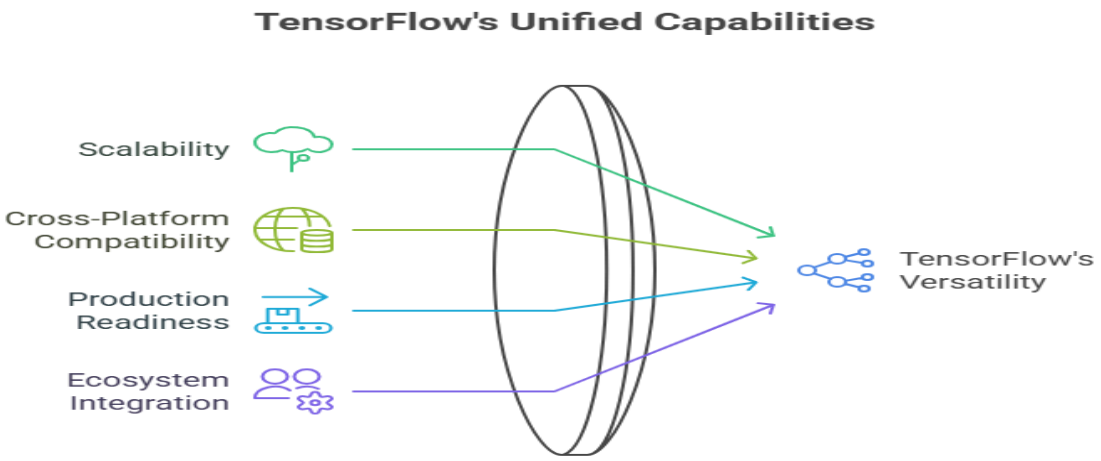
# TensorFlow Basics: A Beginner’s Guide to Deep Learning

## Overview

TensorFlow is an open-source deep learning framework developed by Google Brain. It is designed for building and training machine learning (ML) and deep learning (DL) models at scale.




**Why TensorFlow?**



- Scalable → Can run on CPU, GPU, or TPU.
- Cross-platform → Works on desktops, servers, mobile, and edge devices.
- Production-ready → TensorFlow Serving, TensorFlow Lite, and TensorFlow.js enable deployment anywhere.
- Integration → Strong ecosystem (Keras, TensorBoard, TFX).

Think of TensorFlow as a toolbox for AI engineers, where you can design, train, and deploy neural networks efficiently.

## Key Concepts

🔹 Tensors

- A tensor is the basic data structure in TensorFlow.
- Generalization of scalars, vectors, and matrices.
- **Example:**
    - Scalar → 1
    - Vector → [1,2,3]
    - Matrix → [[1,2],[3,4]]
    - Tensor → [[[1,2],[3,4]], [[5,6],[7,8]]] (3D)

In Deep Learning, images, text, audio → all are represented as tensors.

🔹 Graphs (TF 1.x)

- In TensorFlow 1.x, you first define a computation graph and then execute it in a session.
- Graph = a network of operations (ops) connected by tensors.
- **Example:**

```
import tensorflow as tf
a = tf.constant(5)
b = tf.constant(3)
c = a + b
with tf.Session() as sess:
print(sess.run(c))  # Output: 8
```

- **Drawback** → Hard to debug, not Pythonic.

🔹 Eager Execution (TF 2.x)

- Introduced in TensorFlow 2.x → operations run immediately like standard Python.
- Easier to debug and more intuitive.
- **Example:**

```
import tensorflow as tf
a = tf.constant(5)
b = tf.constant(3)
c = a + b
print(c)  # Output: tf.Tensor(8, shape=(), dtype=int32)
```

TF 2.x uses eager execution by default.

## Building a Simple Neural Network in TensorFlow

We'll use Keras API (high-level) inside TensorFlow.

**Step 1: Import Libraries**

```
import tensorflow as tf

from tensorflow.keras import layers, models
```

**Step 2: Define the Model**

```
model = models.Sequential([

    layers.Dense(128, activation='relu', input_shape=(784,)),  # hidden layer

    layers.Dense(64, activation='relu'),

    layers.Dense(10, activation='softmax')  # output layer (10 classes)

])
```

**Step 3: Compile the Model**

```
model.compile(optimizer='adam',

              loss='sparse_categorical_crossentropy',

              metrics=['accuracy'])
```

**Step 4: Train the Model**

```
# Example with MNIST dataset

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# Flatten images (28x28 → 784)

x_train = x_train.reshape((60000, 784)) / 255.0

x_test = x_test.reshape((10000, 784)) / 255.0

model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))
```

**Step 5: Evaluate**

```
test_loss, test_acc = model.evaluate(x_test, y_test)

print("Test Accuracy:", test_acc)
```

## Key APIs

- tf.keras
  - High-level API for building neural networks quickly.
  - Provides layers, models, optimizers, losses, metrics.

- tf.data
  - Efficient input pipeline.
  - Handles large datasets, streaming, and preprocessing.
  - **Example:**

```
dataset = tf.data.Dataset.from_tensor_slices((x_train, y_train))

dataset = dataset.shuffle(buffer_size=1024).batch(32)
```

- tf.function
  - Converts Python functions into TensorFlow graphs for faster execution.

```
@tf.function

def add(a, b):

return a + b

print(add(tf.constant(3), tf.constant(4)))  # tf.Tensor(7, shape=(), dtype=int32)
```

🔹 TensorBoard

- Visualization tool for training curves, model architecture, embeddings, etc.

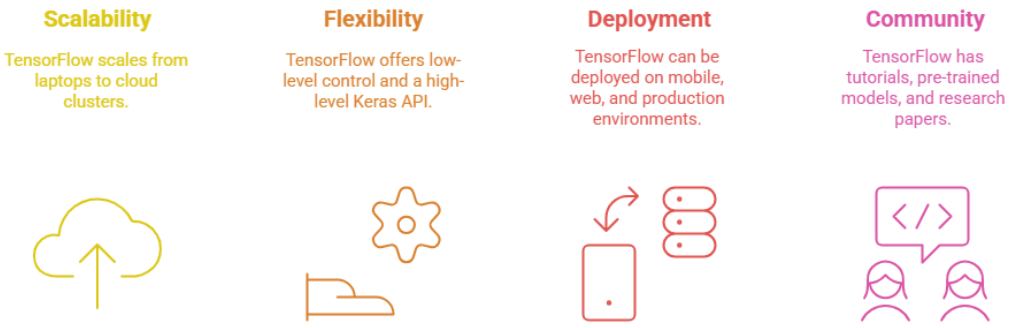
**Example:** MNIST Classification

- **Dataset** → Handwritten digits (0–9).
- **Model** → Fully connected MLP with ReLU + Softmax.
- **Accuracy** → ~98% with simple architecture.

Shows TensorFlow's ease of use for real DL tasks.

## Advantages & Use Cases

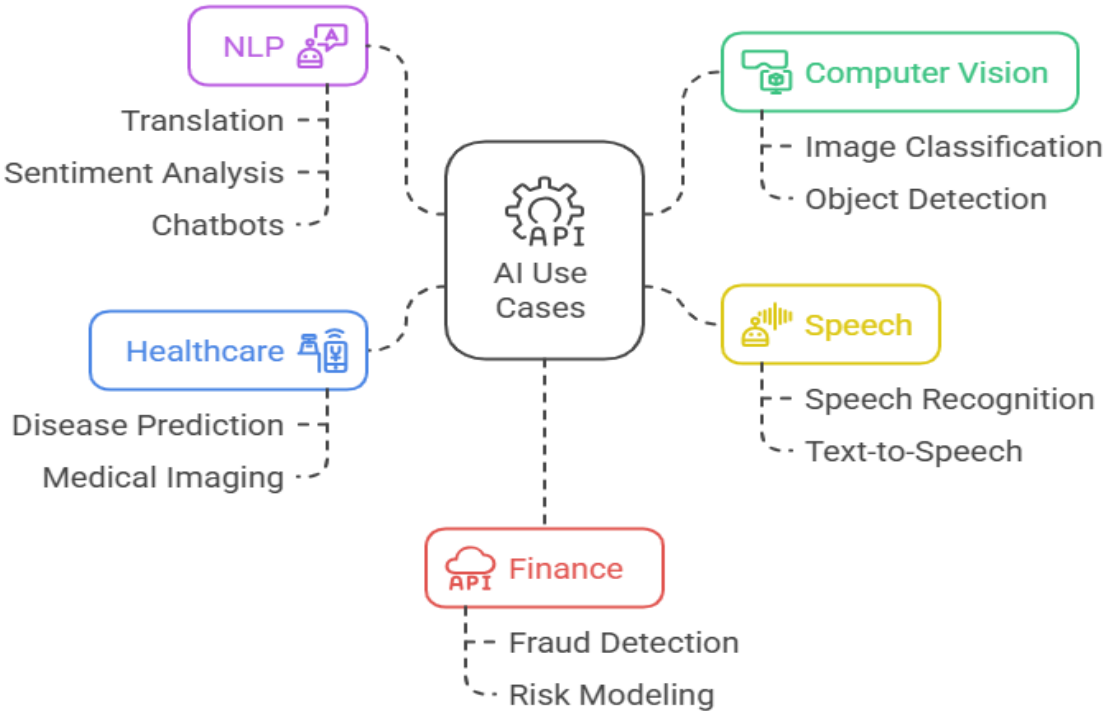
**Advantages:**




- Scalability → From laptops to cloud clusters.
- Flexibility → Low-level control + high-level Keras API.
- Deployment → TF Lite (mobile), TF.js (web), TF Serving (production).

- Community & Ecosystem → Tutorials, pre-trained models, research papers.

## Use Cases




- Computer Vision (image classification, detection).
- NLP (translation, sentiment analysis, chatbots).
- Speech (speech recognition, text-to-speech).
- Healthcare (disease prediction, medical imaging).
- Finance (fraud detection, risk modeling).